

UNIT-VI

DEEP REINFORCEMENT LEARNING:

- Deep Reinforcement Learning Masters Atari Games
- Reinforcement Learning
- Markov Decision Processes (MDP)
- Explore Versus Exploit
- Pole Cart with Policy Gradients
- Q - Learning and Deep Q
- Networks
- Improving and Moving Beyond DQN.

Deep Reinforcement Learning

- Reinforcement learning, which is a branch of machine learning that deals with learning via interaction and feedback.
- Reinforcement learning is essential to building agents that can not only perceive and interpret the world, but also take action and interact with it.
- Discuss how to incorporate deep neural networks into the framework of reinforcement learning and discuss recent advances and improvements in this field.

Deep Reinforcement Learning Masters Atari Games

- The application of deep neural networks to reinforcement learning had a major breakthrough in 2014, when the London startup DeepMind astonished the machine learning community by unveiling a deep neural network that could learn to play Atari games with superhuman skill.
- This network, termed a *Deep Q-Network (DQN)* was the first large-scale successful application of reinforcement learning with deep neural networks.
- DQN was so remarkable because the same architecture, without any changes, was capable of learning 49 different Atari games, despite each game having different rules, goals, and game play structure.

- To accomplish this feat, Deep Mind brought together many traditional ideas in reinforcement learning while also developing a few novel techniques that proved key to DQN's success.
- Implement DQN, as it is described in the *Nature paper* “Human-level control through deep reinforcement learning.”
- But first, let's take a dive into reinforcement learning (Figure 9-1).



Figure 9-1. A deep reinforcement learning agent playing Breakout. This image is from the OpenAI Gym3 DQN agent that we build in this chapter.

What Is Reinforcement Learning?

- Reinforcement learning, at its essentials, is learning by interacting with an environment.
- Learning process involves an *actor*, an *environment*, and a *reward signal*.
- The actor chooses to take an action in the environment, for which the actor is rewarded accordingly.
- The way in which an actor chooses actions is called a *policy*.
- *The* actor wants to increase the reward it receives, and so must learn an optimal policy for interacting with the environment (Figure 9-2).
- Reinforcement learning is different from the other types of learning that we have covered thus far.
- In traditional supervised learning, we are given data and labels, and are tasked with predicting labels given data.

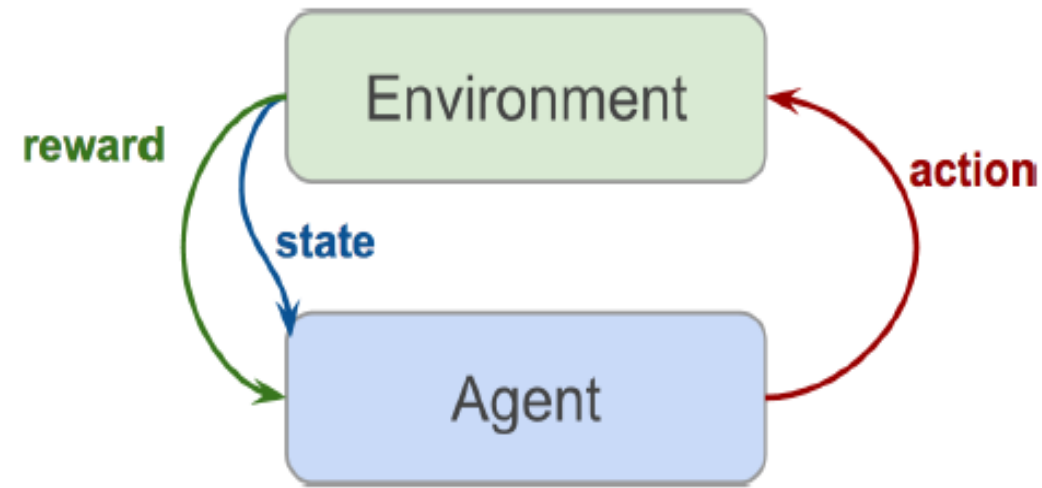


Figure 9-2. Reinforcement learning setup

- In unsupervised learning, we are given just data and are tasked with discovering underlying structure in this data.
- In reinforcement learning, we are given neither data nor labels.
- Learning signal is derived from the rewards given to the agent by the environment.

- Reinforcement learning is exciting to many in the artificial intelligence community because it is a general-purpose framework for creating intelligent agents.
- Given an environment and some rewards, the agent learns to interact with that environment to maximize its total reward.
- This type of learning is more in line with how humans develop.
- Yes, we can build a pretty good model to classify dogs from cats with extremely high accuracy by training on thousands of images.
- But you won't find this approach used in any elementary schools.
- Humans interact with their environment to learn representations of the world which they can use to make decisions.
- Reinforcement learning applications are at the forefront of many cutting-edge technologies including self-driving cars, robotic motor control, game playing, air-conditioning control, ad-placement optimization, and stock market trading strategies.

- As an illustrative exercise, we'll be tackling a simple reinforcement learning and control problem called pole-balancing.
- In this problem, there is a cart with a pole that is connected by a hinge, so the pole can swing around the cart.
- There is an agent that can control the cart, moving it left or right.
- There is an environment, which rewards the agent when the pole is pointed upward, and penalizes the agent when the pole falls over (Figure 9-3).

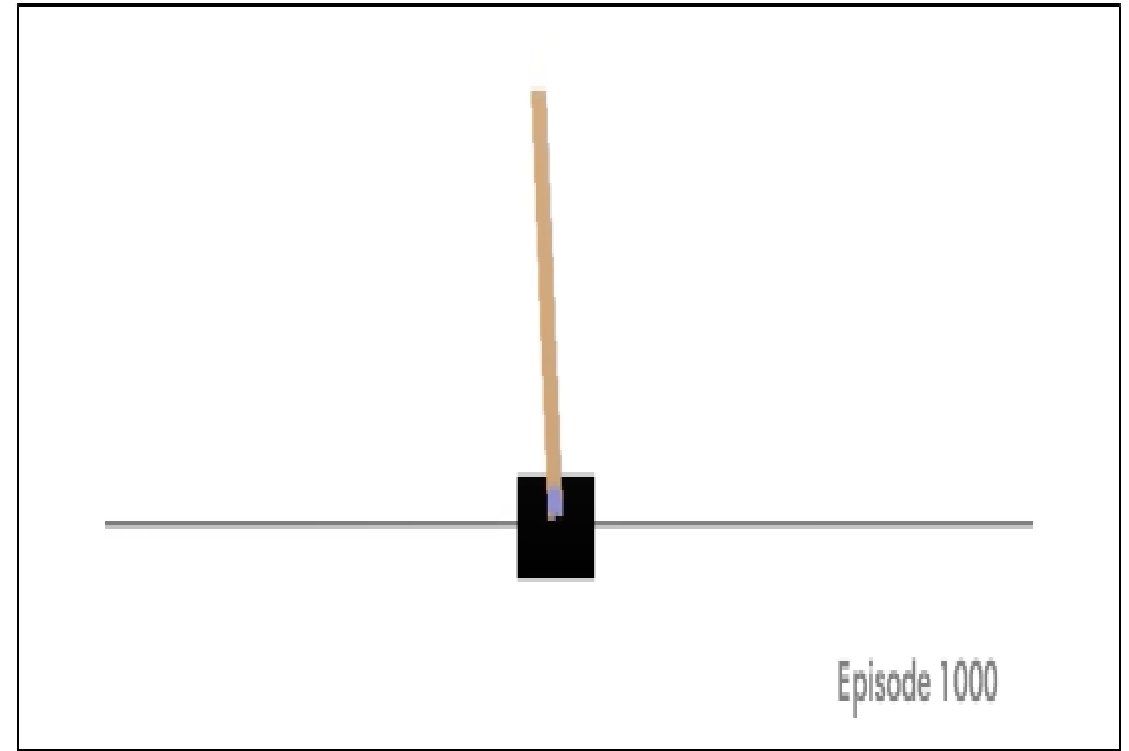


Figure 9-3. A simple reinforcement learning agent balancing a pole. This image is from our OpenAI Gym Policy Gradient agent that we build in this chapter.

Markov Decision Processes (MDP)

- Our pole-balancing example has a few important elements, which we formalize as a *Markov Decision Process (MDP)*. These elements are:

State

- The cart has a range of possible places on the x-plane where it can be. Similarly, the pole has a range of possible angles.

Action

- The agent can take action by moving the cart either left or right.

State Transition

- When the agent acts, the environment changes—the cart moves and the pole changes angle and velocity.

Reward

- If an agent balances the pole well, it receives a positive reward. If the pole falls,
- the agent receives a negative reward.

An MDP is defined as the following:

- S , a finite set of possible states
- A , a finite set of actions
- P, r, s', a , a state transition function
- R , reward function

- MDPs offer a mathematical framework for modeling decision-making in a given environment (Figure 9-4).
- As an agent takes action in an MDP framework, it forms an *episode*.
- *An episode consists of series of tuples of states, actions, and rewards.*
- Episodes run until the environment reaches a terminal state, like the “Game Over” screen in Atari games, or when the pole hits the ground in our pole-cart example.
- The following equation shows the variables in an episode:
- $s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_n, a_n, r_n$
- In pole-cart, our environment state can be a tuple of the position of the cart and the angle of the pole, like so:
- $(x_{cart}, \vartheta_{pole})$.

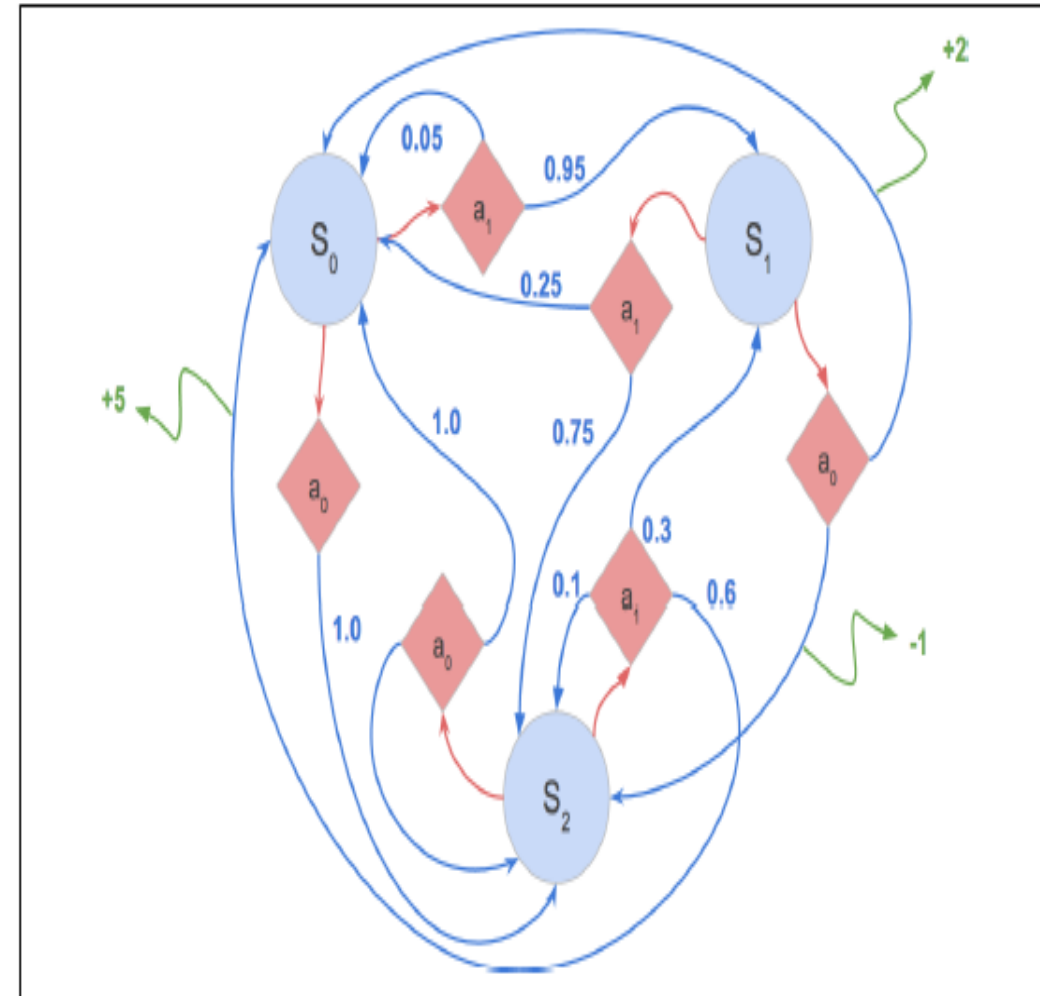


Figure 9-4. An example of an MDP. Blue circles represent the states of the environment. Red diamonds represent actions that can be taken. The edges from diamonds to circles represent the transition from one state to the next. The numbers along these edges represent the probability of taking a certain action. The numbers at the end of the green arrows represent the reward given to the agent for making the given transition.

POLICY

- MDP's aim is to find an optimal policy for our agent.
- *Policies are the way in which* our agent acts based on its current state.
- Formally, policies can be represented as a function π *that chooses the action a that the agent will take in state s .*
- The objective of our MDP is to find a policy to maximize the expected future return:
- $\max_{\pi} E[R_0 + R_1 + \dots R_t \mid \pi]$
- In this objective, R represents the *future return of each episode*.
- *Let's define exactly* what future return means.

Future Return

- Future return is how we consider the rewards of the future.
- Choosing the best action requires consideration of not only the immediate effects of that action, but also the long-term consequences.
- Sometimes the best action actually has a negative immediate effect, but a better long-term result.
- Ex: a mountain-climbing agent that is rewarded by its altitude may actually have to climb downhill to reach a better path to the mountain's peak.
- Therefore, we want our agents to optimize for *future return*.
- *In order to do that, the agent must consider the future consequences of its actions.*

- For example, in a game of Pong, the agent receives a reward when the ball passes into the opponent's goal.
- However, the actions responsible for this reward (the inputs that position the racquet to strike scoring hit) happen many time steps before the reward is received.
- The reward for each of those actions is delayed.
- We can incorporate delayed rewards into our overall reward signal by constructing *a return for each time step that takes into account future rewards as well as immediate rewards*.

Explore Versus Exploit

- Reinforcement learning is fundamentally a trial-and-error process.
- In such a framework, an agent afraid to make mistakes can prove to be highly problematic. Consider the following scenario.
- A mouse is placed in the maze shown in Figure 9-5.
- Our agent must control the mouse to maximize reward.

- If the mouse gets the water, it receives a reward of +1; if the mouse reaches a poison container (red), it receives a reward of -10; if the mouse gets the cheese, it receives a reward of +100.
- Upon receiving reward, the episode is over.
- The optimal policy involves the mouse successfully navigating to the cheese and eating it.
- In the first episode, the mouse takes the left route, steps on a trap, and receives a -10 reward.
- In the second episode, the mouse avoids the left path, since it resulted in such a negative reward, and drinks the water immediately to its right for a +1 reward.
- After two episodes, it would seem that the mouse has found a good policy.
- It continues to follow its learned policy on subsequent episodes and achieves the moderate +1 reward reliably.

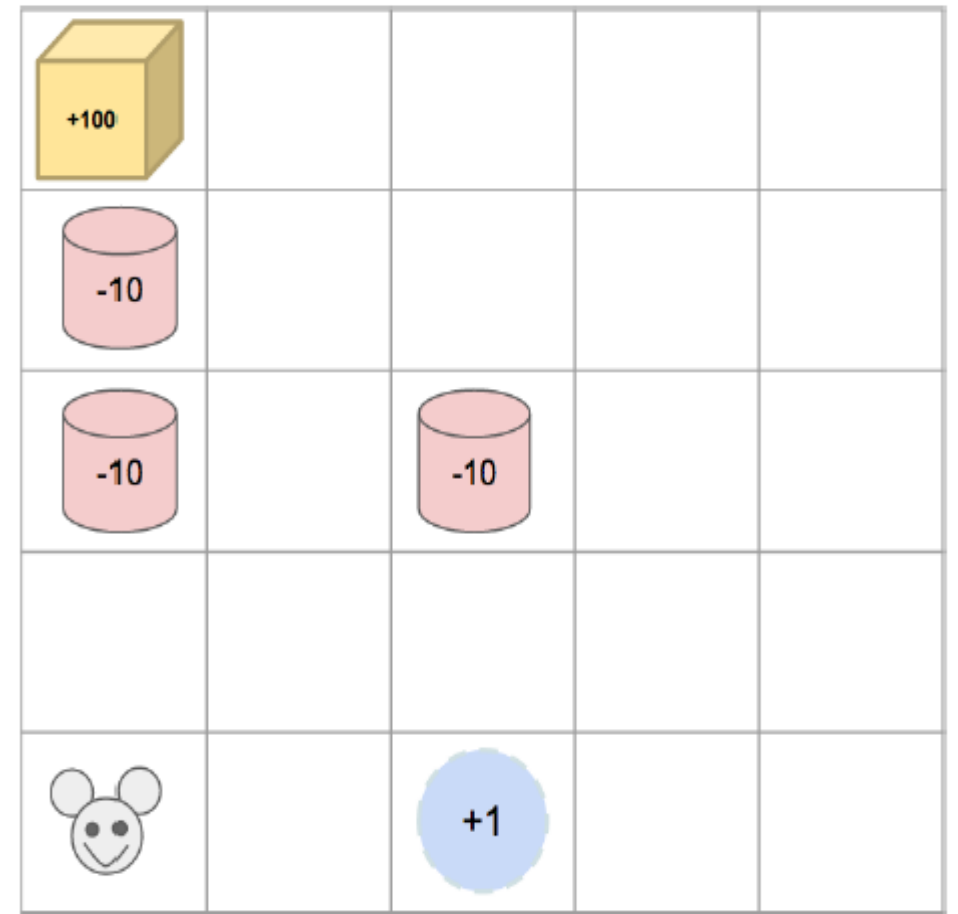


Figure 9-5. A predicament that many mice find themselves in

Since our agent utilizes a greedy strategy—always choosing the model’s best action—it is stuck in a policy that is a *local maximum*

Pole-Cart with Policy Gradients

- We're going to implement a policy-gradient agent to solve pole-cart, a classic reinforcement learning problem.
- We will be using an environment from the OpenAI Gym created just for this task.
- OpenAI Gym The OpenAI Gym is a Python toolkit for developing reinforcement agents.
- OpenAI Gym provides an easy-to-use interface for interacting with a variety of environments.
- It contains over 100 open-source implementations of common reinforcement learning environments.
- OpenAI Gym speeds up development of reinforcement learning agents by handling everything on the environment simulation side, allowing researchers to focus on their agent and learning algorithms.
- Another benefit of OpenAI Gym is that researchers can fairly compare and evaluate their results with others because they can all use the same standardized environment for a task.
- We'll be using the pole-cart environment from OpenAI Gym to create an agent that can easily interact with this environment

Creating an Agent

- To create an agent that can interact with an OpenAI environment, we'll define a class PGAgent, which will contain our model architecture, model weights, and hyper parameters

Policy Gradient Main Function

- Lets put this all together in our main function, which will create an OpenAI Gym environment for CartPole, make an instance of our agent, and have our agent interact with and train on the CartPole environment

PGAgent Performance on Pole-Cart

- Figure 9-6 is a chart of the average reward of our agent at each step of training.
- We try out 8 different sampling methods, and achieve best results with epsilon greedy annealing from 1.0 to 0.001.

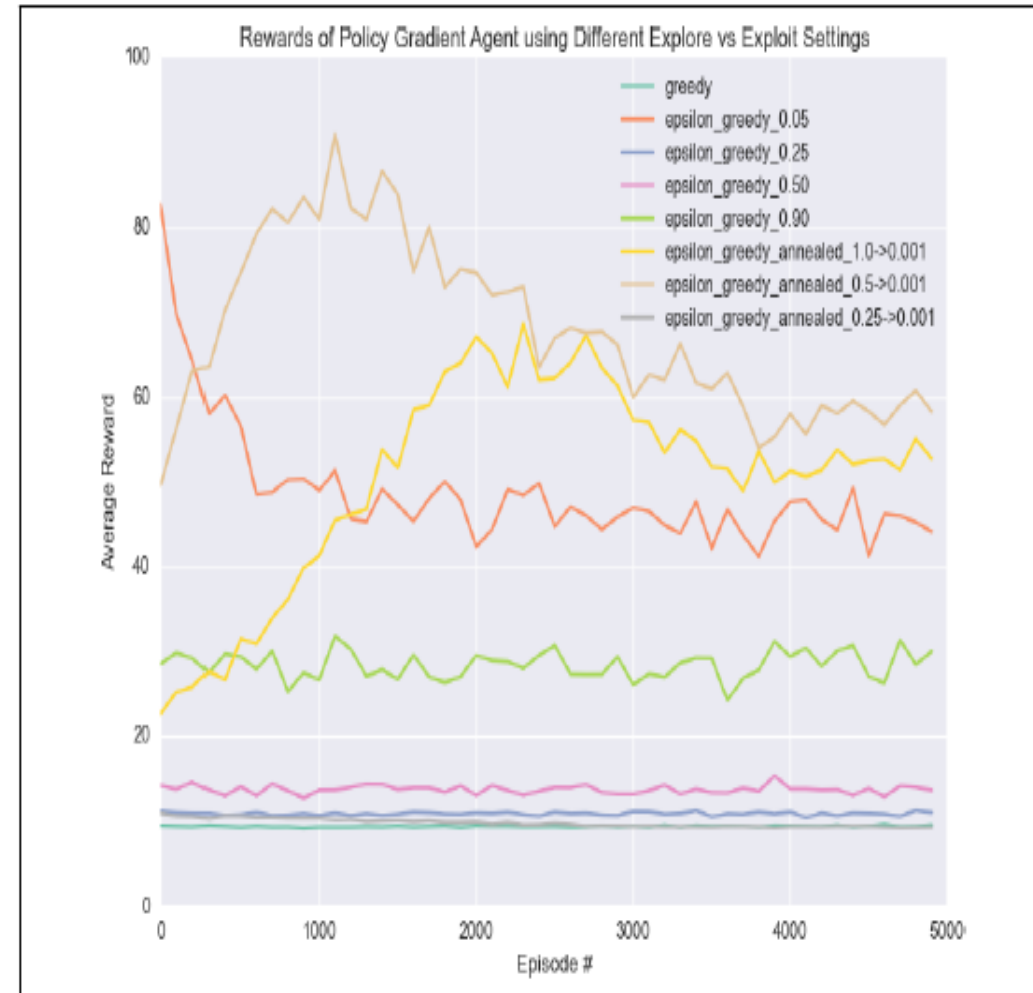


Figure 9-6. Explore-exploit configurations affect how fast and how well learning occurs

Q-Learning and Deep Q-Networks

- Q-learning is in the category of reinforcement learning called value-learning.
- Instead of directly learning a policy, we will be learning the value of states and actions.
- Qlearning involves learning a function, a *Q-function*, which represents the quality of a state, action pair.
- The Q-function, defined $Q(s, a)$, is a function that calculates the maximum discounted future return when action a is performed in state s .
- The Q-value represents our expected long-term rewards, given we are at a state, and take an action, and then take every subsequent action perfectly (to maximize expected future reward).
- This can be expressed formally as: $Q^*(s_t, a_t) = \max_{\pi} E\left[\sum_{i=t}^T \gamma^i r^i\right]$

- It is difficult, even for humans, to know how good an action is, because you need to know how you are going to act in the future.
- Our expected future returns depend on what our long-term strategy is going to be.
- This seems to be a bit of a chicken-and-egg problem.
- In order to value a state, action pair you need to know all the perfect subsequent actions.
- And in order to know the best actions, you need to have accurate values for a state and action.
- The Bellman Equation We solve this dilemma by defining our Q-values as a function of future Q-values.
- This relation is called the *Bellman equation*, and it states that the *maximum future* reward for taking action is the current reward plus the next step's max future reward from taking the next action a' :

$$Q^*(s_t, a_t) = E[r_t + \gamma \max_{a'} Q^*(s_{t+1}, a')]$$

- And since we can now relate between Q-values past and future, this equation conveniently defines an update rule.
- Namely, we can update past Q-values to be based on future Q-values.
- This is powerful because there exists a Q-value we know is correct: the Q-value of the very last action before the episode is over.
- For this last state, we know exactly that the next action led to the next reward, so we can perfectly set the Q-values for that state.
- We can use the update rule, then, to propagate that Q-value to the previous time step:

$$\widehat{Q}_j \rightarrow \widehat{Q}_{j+1} \rightarrow \widehat{Q}_{j+2} \rightarrow \dots \rightarrow Q^*$$

- This updating of the Q-value is known as *value iteration*.
- Our first Q-value starts out completely wrong, but this is perfectly acceptable.
- With each iteration, we can update our Q-value via the correct one from the future.
- After one iteration, the last Q-value is accurate, since it is just the reward from the last state and action before episode termination.
- Then we perform our Q-value update, which sets the second-to-last Q-value.